```
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from tensorflow.keras.datasets import fashion_mnist
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D
from tensorflow.keras.layers import Dense, Flatten, Dropout
from tensorflow.keras.utils import to_categorical

from sklearn.metrics import confusion_matrix
from sklearn.metrics import classification_report

# Load Dataset
(X_train, y_train), (X_test, y_test) = fashion_mnist.load_data()

print("Training Images:", X_train.shape)
print("Testing Images:", X_test.shape)

# Normalize pixel values
X_train = X_train.astype('float32') / 255.0
X_test = X_test.astype('float32') / 255.0

# Reshape for CNN
X_train = X_train.reshape(-1,28,28,1)
X_test = X_test.reshape(-1,28,28,1)

# One-hot encoding
y_train_cat = to_categorical(y_train, 10)
y_test_cat = to_categorical(y_test, 10)

class_names = [
    "T-shirt",
    "Trouser",
    "Pullover",
    "Dress",
    "Coat",
    "Sandal",
    "Shirt",
    "Sneaker",
    "Bag",
    "Ankle Boot"
]

model = Sequential([

    Conv2D(32, (3,3),
           activation='relu',
           input_shape=(28,28,1)),
```

```
    MaxPooling2D((2,2)),

    Conv2D(64, (3,3),
           activation='relu'),

    MaxPooling2D((2,2)),

    Conv2D(128, (3,3),
           activation='relu'),

    Flatten(),

    Dense(128,
          activation='relu'),

    Dropout(0.5),

    Dense(10,
          activation='softmax')
])


model.compile(
    optimizer='adam',
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

model.summary()

history = model.fit(
    X_train,
    y_train_cat,
    epochs=15,
    batch_size=128,
    validation_split=0.2
)


test_loss, test_acc = model.evaluate(
    X_test,
    y_test_cat
)

print("\nTest Accuracy:", test_acc)

# Accuracy Curve
plt.figure(figsize=(12,5))

plt.subplot(1,2,1)
plt.plot(history.history['accuracy'],
```

```
         label='Train Accuracy')
plt.plot(history.history['val_accuracy'],
         label='Validation Accuracy')
plt.title("Accuracy Curve")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.legend()

# Loss Curve
plt.subplot(1,2,2)
plt.plot(history.history['loss'],
         label='Train Loss')
plt.plot(history.history['val_loss'],
         label='Validation Loss')
plt.title("Loss Curve")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.legend()

plt.show()

# Predictions
y_pred_prob = model.predict(X_test)

y_pred = np.argmax(
    y_pred_prob,
    axis=1
)

# Confusion Matrix
cm = confusion_matrix(
    y_test,
    y_pred
)

plt.figure(figsize=(10,8))

sns.heatmap(
    cm,
    annot=True,
    fmt='d',
    cmap='Blues',
    xticklabels=class_names,
    yticklabels=class_names
)

plt.title("Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()
```

```
# Classification Report
print(
    classification_report(
        y_test,
        y_pred,
        target_names=class_names
    )
)

# Find Most Misclassified Class

misclassified = cm.sum(axis=1) - np.diag(cm)

for cls, count in zip(class_names,
                      misclassified):
    print(f"{cls}: {count}")

max_index = np.argmax(misclassified)

print("\nMost Misclassified Class:")
print(class_names[max_index])
print("Misclassified Samples:",
      misclassified[max_index])

# Show Sample Misclassified Images

mis_idx = np.where(
    y_test != y_pred
)[0]

plt.figure(figsize=(12,8))

for i in range(9):

    idx = mis_idx[i]

    plt.subplot(3,3,i+1)

    plt.imshow(
        X_test[idx].reshape(28,28),
        cmap='gray'
    )

    plt.title(
        f"True:{class_names[y_test[idx]]}\n"
        f"Pred:{class_names[y_pred[idx]]}"
    )

    plt.axis('off')
```

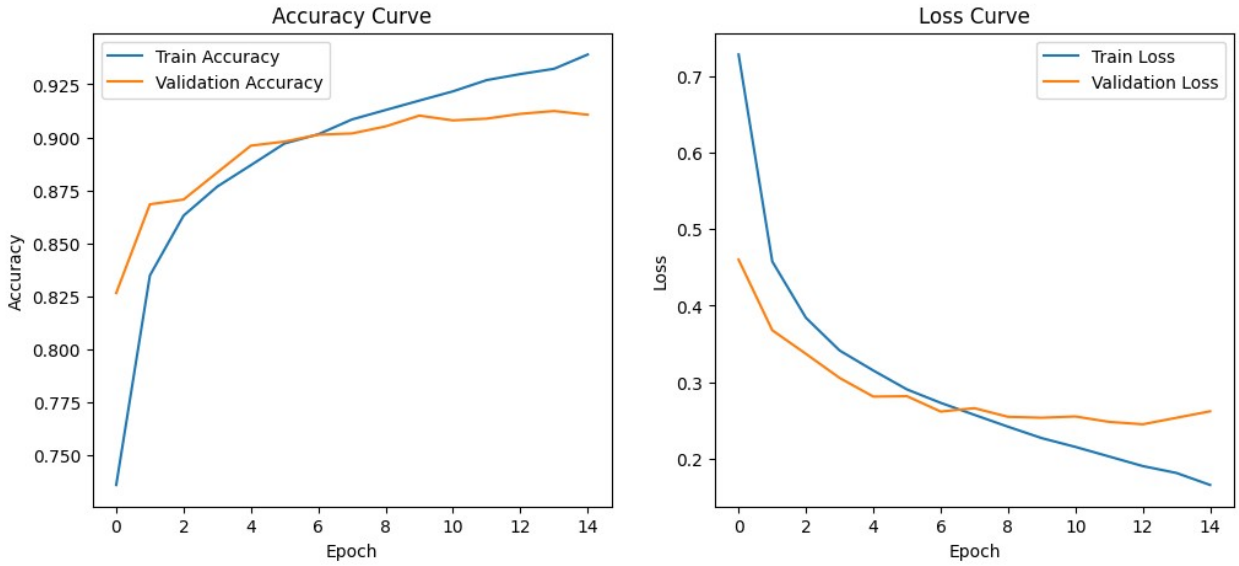
```
plt.tight_layout()
plt.show()
```

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-
keras-datasets/train-labels-idx1-ubyte.gz
29515/29515 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
Downloading data from https://storage.googleapis.com/tensorflow/tf-
keras-datasets/train-images-idx3-ubyte.gz
26421880/26421880 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
Downloading data from https://storage.googleapis.com/tensorflow/tf-
keras-datasets/t10k-labels-idx1-ubyte.gz
5148/5148 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
Downloading data from https://storage.googleapis.com/tensorflow/tf-
keras-datasets/t10k-images-idx3-ubyte.gz
4422102/4422102 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
Training Images: (60000, 28, 28)
Testing Images: (10000, 28, 28)

/usr/local/lib/python3.12/dist-packages/keras/src/layers/
convolutional/base_conv.py:113: UserWarning: Do not pass an
`input_shape`/`input_dim` argument to a layer. When using Sequential
models, prefer using an `Input(shape)` object as the first layer in
the model instead.
  super().__init__(activity_regularizer=activity_regularizer,
**kwargs)

Model: "sequential"
```

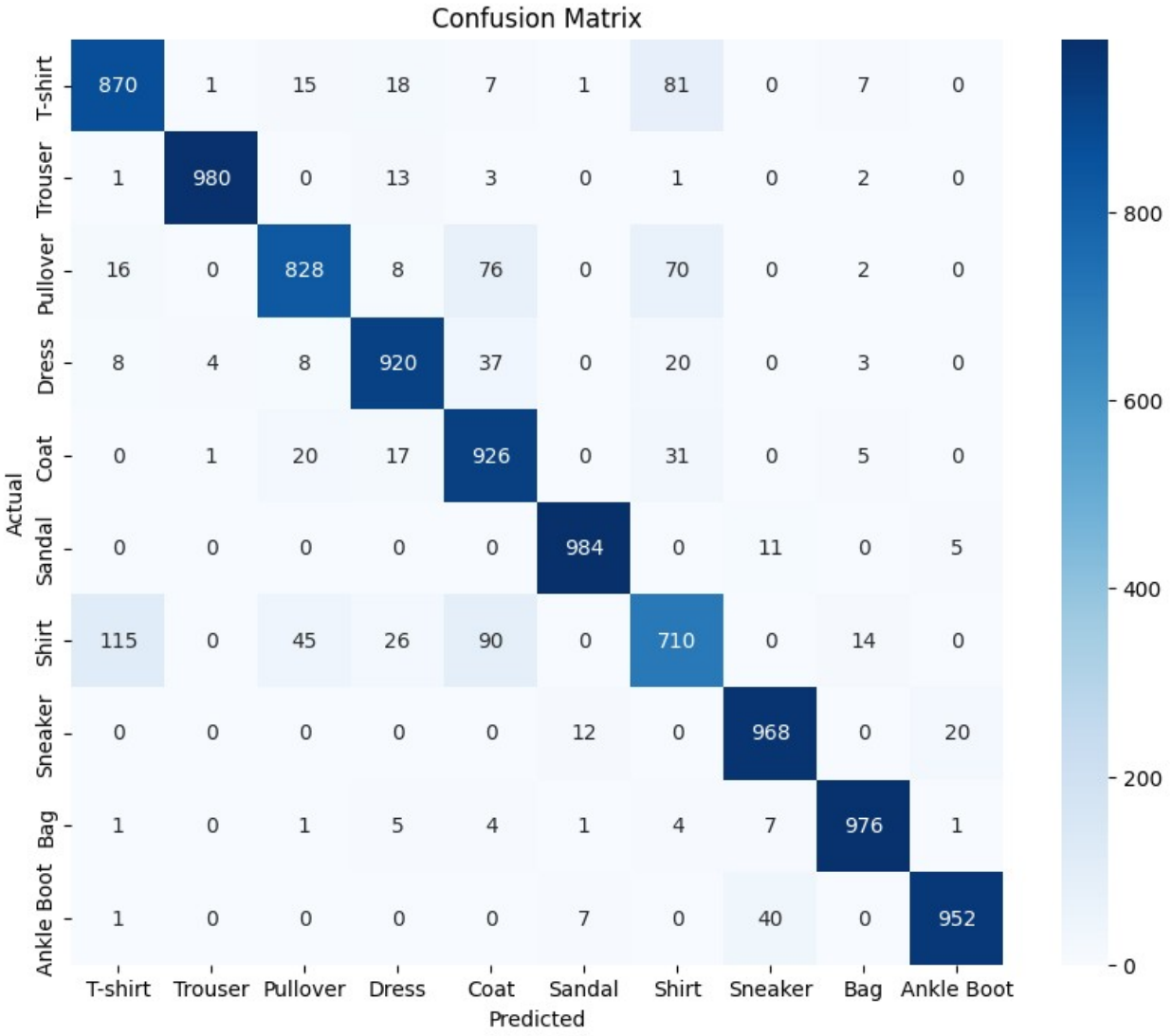
```
┏━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━┳━━━━━━━━━━━━━━━━━━━━━━━━┳━━━━━━━━━━━━━━━┓
┃ Layer (type)                    ┃ Output Shape           ┃       Param # ┃
┡━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━╇━━━━━━━━━━━━━━━━━━━━━━━━╇━━━━━━━━━━━━━━━┩
│ conv2d (Conv2D)                 │ (None, 26, 26, 32)     │           320 │
├─────────────────────────────────┼────────────────────────┼───────────────┤
│ max_pooling2d (MaxPooling2D)    │ (None, 13, 13, 32)     │             0 │
├─────────────────────────────────┼────────────────────────┼───────────────┤
│ conv2d_1 (Conv2D)               │ (None, 11, 11, 64)     │        18,496 │
├─────────────────────────────────┼────────────────────────┼───────────────┤
│ max_pooling2d_1 (MaxPooling2D)  │ (None, 5, 5, 64)       │             0 │
├─────────────────────────────────┼────────────────────────┼───────────────┤
```

```
| conv2d_2 (Conv2D)                    | (None, 3, 3, 128)       |
73,856 |
├──────────────────────────────────────┼─────────────────────────┼──────────
──────┤
| flatten (Flatten)                    | (None, 1152)            |
0 |
├──────────────────────────────────────┼─────────────────────────┼──────────
──────┤
| dense (Dense)                        | (None, 128)             |
147,584 |
├──────────────────────────────────────┼─────────────────────────┼──────────
──────┤
| dropout (Dropout)                    | (None, 128)             |
0 |
├──────────────────────────────────────┼─────────────────────────┼──────────
──────┤
| dense_1 (Dense)                      | (None, 10)              |
1,290 |
└──────────────────────────────────────┴─────────────────────────┴──────────
──────┘

 Total params: 241,546 (943.54 KB)

 Trainable params: 241,546 (943.54 KB)

 Non-trainable params: 0 (0.00 B)

Epoch 1/15
375/375 ━━━━━━━━━━━━━━━━━━━━ 8s 8ms/step - accuracy: 0.7360 - loss:
0.7281 - val_accuracy: 0.8266 - val_loss: 0.4602
Epoch 2/15
375/375 ━━━━━━━━━━━━━━━━━━━━ 2s 6ms/step - accuracy: 0.8349 - loss:
0.4578 - val_accuracy: 0.8684 - val_loss: 0.3680
Epoch 3/15
375/375 ━━━━━━━━━━━━━━━━━━━━ 2s 6ms/step - accuracy: 0.8631 - loss:
0.3842 - val_accuracy: 0.8707 - val_loss: 0.3371
Epoch 4/15
375/375 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step - accuracy: 0.8768 - loss:
0.3415 - val_accuracy: 0.8834 - val_loss: 0.3056
Epoch 5/15
375/375 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step - accuracy: 0.8869 - loss:
0.3154 - val_accuracy: 0.8961 - val_loss: 0.2813
Epoch 6/15
375/375 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step - accuracy: 0.8971 - loss:
0.2907 - val_accuracy: 0.8980 - val_loss: 0.2818
Epoch 7/15
375/375 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step - accuracy: 0.9014 - loss:
0.2731 - val_accuracy: 0.9013 - val_loss: 0.2617
Epoch 8/15
375/375 ━━━━━━━━━━━━━━━━━━━━ 2s 6ms/step - accuracy: 0.9084 - loss:
```

```
0.2575 - val_accuracy: 0.9018 - val_loss: 0.2662
Epoch 9/15
375/375 ━━━━━━━━━━━━━━━━━━━━ 2s 6ms/step - accuracy: 0.9129 - loss:
0.2420 - val_accuracy: 0.9052 - val_loss: 0.2549
Epoch 10/15
375/375 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step - accuracy: 0.9173 - loss:
0.2271 - val_accuracy: 0.9103 - val_loss: 0.2537
Epoch 11/15
375/375 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step - accuracy: 0.9217 - loss:
0.2156 - val_accuracy: 0.9080 - val_loss: 0.2553
Epoch 12/15
375/375 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step - accuracy: 0.9270 - loss:
0.2031 - val_accuracy: 0.9088 - val_loss: 0.2483
Epoch 13/15
375/375 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step - accuracy: 0.9299 - loss:
0.1906 - val_accuracy: 0.9111 - val_loss: 0.2451
Epoch 14/15
375/375 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step - accuracy: 0.9323 - loss:
0.1816 - val_accuracy: 0.9124 - val_loss: 0.2536
Epoch 15/15
375/375 ━━━━━━━━━━━━━━━━━━━━ 2s 6ms/step - accuracy: 0.9391 - loss:
0.1659 - val_accuracy: 0.9107 - val_loss: 0.2621
313/313 ━━━━━━━━━━━━━━━━━━━━ 2s 4ms/step - accuracy: 0.9114 - loss:
0.2718

Test Accuracy: 0.9114000201225281
```




```
313/313 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step
```



	precision	recall	f1-score	support
T-shirt	0.86	0.87	0.86	1000
Trouser	0.99	0.98	0.99	1000
Pullover	0.90	0.83	0.86	1000
Dress	0.91	0.92	0.92	1000
Coat	0.81	0.93	0.86	1000
Sandal	0.98	0.98	0.98	1000
Shirt	0.77	0.71	0.74	1000
Sneaker	0.94	0.97	0.96	1000
Bag	0.97	0.98	0.97	1000
Ankle Boot	0.97	0.95	0.96	1000
accuracy			0.91	10000
macro avg	0.91	0.91	0.91	10000
weighted avg	0.91	0.91	0.91	10000

```
T-shirt: 130
Trouser: 20
Pullover: 172
Dress: 80
Coat: 74
Sandal: 16
Shirt: 290
Sneaker: 32
Bag: 24
Ankle Boot: 48

Most Misclassified Class:
Shirt
Misclassified Samples: 290
```